ENTERPRISE DEVSECOPS METHODOLOGY DOCUMENT

Application Security | CI/CD Security | Container Hardening | Kubernetes Security | GitOps | Supply Chain | Runtime Detection | Observability

Author: Jagriti Banerjee

Build	Break	Detect	Harden	Retest	Report

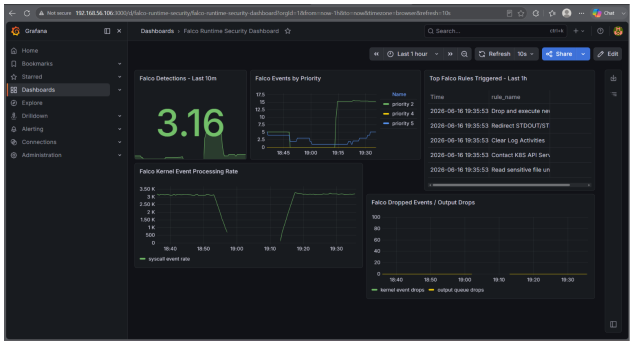
Field	Details
Project	Enterprise DevSecOps Red Team and Supply Chain Security Lab
Methodology Model	Build -> Break -> Detect -> Harden -> Retest -> Report
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, ArgoCD, Falco, Prometheus, Grafana
Assessment Boundary	Controlled local lab only; no external or unauthorized targets

Professional Scope	This methodology explains how the project was planned, built, tested, hardened, validated, and documented using an enterprise-style DevSecOps assessment lifecycle. It is written as a portfolio-ready methodology for DevSecOps, Kubernetes security, cloud security, application security, and security engineering roles.

Ethical Boundary	All attack simulations, vulnerable application testing, privileged pod demonstrations, RBAC escalation scenarios, and runtime detections are performed inside a private VM lab. The methodology excludes public internet exploitation and unauthorized third-party testing.

Methodology Coverage Snapshot

Domain	Coverage
Application Security	Vulnerable Flask routes, manual validation, SAST, dependency scanning
Container Security	Multi-stage Dockerfile, non-root user, healthcheck, Trivy image scan
Kubernetes Security	SecurityContext, RBAC, NetworkPolicy, Pod Security, admission control
Delivery Governance	GitHub Actions, GHCR, ArgoCD sync, drift detection, self-healing
Supply Chain	Cosign signing, Syft SBOM, SBOM attestation, SLSA-style provenance
Runtime and Observability	Falco, Falcosidekick, Prometheus, Grafana, alert rules



Evidence snapshot: custom Grafana runtime security dashboard included in the submitted project archive.

Table of Contents

Archive Basis	The methodology was aligned to the submitted project archive. The archive contains 159 evidence screenshots, 0 security report artifacts, 0 GitHub Actions workflow file(s), and 0 Kubernetes YAML manifest(s), plus monitoring, policy, supply-chain, and runtime-security configuration files.

Document Layout Note

The document is intentionally spaced for review rather than compressed for maximum page density. Tables, callouts, and screenshots are balanced so the methodology remains readable on screen and in PDF review.

Reviewer Navigation Guide

Reviewer Goal	Where to Look
Application security proof	Section 3 and screenshot evidence for SQL injection, command injection, Bandit, and pip-audit.
CI/CD and Docker proof	Section 4, workflow evidence, Docker hardening controls, and Trivy image scan evidence.
Kubernetes security proof	Section 5, RBAC checks, NetworkPolicy tests, SecurityContext proof, and read-only filesystem evidence.
Governance proof	Section 6 for Gatekeeper admission control and ArgoCD drift/self-healing evidence.
Supply-chain proof	Section 7 for Cosign signing, Syft SBOM, attestation, and provenance evidence.
Detection and monitoring proof	Sections 8 and 9 for Falco, Prometheus, Grafana, and alert rule evidence.

1. Methodology Overview and Assessment Principles

The project follows an enterprise-style DevSecOps assessment methodology designed to demonstrate the full movement of an application through insecure build conditions, security testing, hardening, deployment governance, runtime monitoring, and formal reporting. The methodology does not stop at finding vulnerabilities; it documents validation, remediation, retesting, and operational controls.

Plan	Build	Break	Detect	Harden	Retest	Report

Lifecycle Stage	Methodology Intent	Primary Evidence
Plan	Define private-lab scope, safety boundaries, expected phases, and evidence targets.	Project folders, README, evidence index
Build	Create vulnerable app, Docker image, Kubernetes manifests, workflow files, and monitoring configs.	Source code, Dockerfile, YAML manifests
Break	Safely validate SQL injection, command injection, privileged pod access, RBAC escalation, and GitOps drift.	curl output, kubectl output, screenshots
Detect	Use scanners and runtime tools to detect weak code, dependencies, images, manifests, runtime behavior, and metrics.	Bandit, pip-audit, Trivy, Falco, Prometheus
Harden	Apply non-root execution, RBAC, NetworkPolicy, Pod Security, Gatekeeper, image signing, and GitOps controls.	Hardened manifests and policy files
Retest	Repeat high-risk tests after controls are applied to confirm that risk is reduced.	Before/after proof screenshots
Report	Package the work into a professional methodology, findings report, and evidence-backed project summary.	PDFs, markdown docs, screenshot index

Assessment Principles

Principle	Application in the Project
Controlled Scope	All tests stay inside the private VM and local Kubernetes cluster. No third-party systems are targeted.
Evidence First	Every major activity produces screenshots, command output, scanner reports, or configuration files.
Manual Validation	Findings are confirmed through safe proof-of-concept commands instead of accepting scanner output blindly.
Defense in Depth	Controls are applied across code, dependency, container, Kubernetes, admission, GitOps, runtime, and observability layers.
Retest Discipline	Every critical remediation is followed by validation showing the insecure action is blocked or corrected.

2. Lab Environment, Repository Structure, and Evidence Baseline

The environment methodology begins by confirming that the private VM can support container and Kubernetes workloads. Because the lab is resource-constrained, the approach favors a lightweight Kind cluster, local-only access, controlled namespaces, and selective scaling of heavy components.

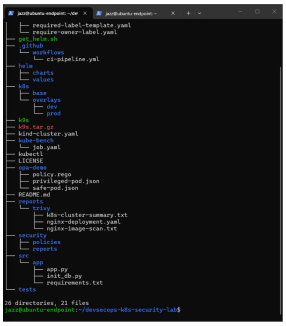
Area	Methodology
VM Baseline	Validate CPU, memory, swap, and disk space before installing heavy tools.
Tool Order	Install Docker, kubectl, Helm, Kind, K9s, Trivy, Cosign, Syft, Falco, and monitoring components in controlled phases.
Cluster Strategy	Use Kind for Kubernetes simulation; use Calico when real NetworkPolicy enforcement evidence is required.
Namespace Separation	Use dedicated namespaces such as devsecops, argocd, falco, monitoring, runtime-lab, and rbac-lab.
Access Model	Expose services only with localhost port-forwarding or Kind NodePort mappings inside the VM.
Resource Control	Scale down ArgoCD or monitoring components when moving to heavy runtime phases to preserve VM stability.

Repository Methodology

The repository is organized so that a reviewer can understand the project without executing the lab. Application code, Kubernetes manifests, GitOps configuration, monitoring files, supply-chain artifacts, runtime simulations, reports, and screenshots are separated into clear folders.

Folder	Purpose
src/app	Flask application, database initialization, and dependency requirements.
.github/workflows	Security pipeline definition for SAST, SCA, Trivy, Docker build, and registry push.
k8s/base	Namespace, Deployment, Service, RBAC, NetworkPolicy, and Kustomize manifests.
gitops/argocd	ArgoCD Application manifest for GitOps deployment.
security	Cosign public key, SBOMs, provenance, policies, and reports.
redteam	Controlled runtime and RBAC attack simulation manifests.

Folder	Purpose
monitoring	Prometheus, Grafana, Falco, Falcosidekick dashboards and rules.
screenshots	Visual evidence collected during implementation and validation.



Evidence snapshot: project folder structure created for a recruiter-readable DevSecOps portfolio.

3. Vulnerable Application Build and Manual Attack Validation

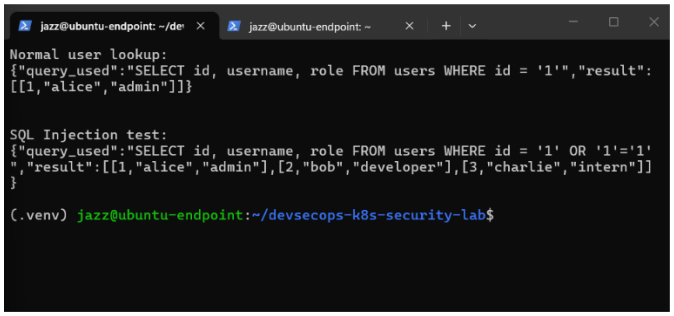
The application methodology intentionally creates a small vulnerable Flask service so that application security risks can be demonstrated without touching external systems. The vulnerable routes allow safe testing of SQL injection, command injection, and unsafe template rendering patterns inside the local lab.

Component	Methodology Purpose	Validation
/health	Provide stable runtime and Kubernetes probe endpoint.	curl returns status ok.
/user?id=	Demonstrate insecure SQL query construction with user input.	OR 1=1 style payload returns multiple rows.
/ping?host=	Demonstrate unsafe shell execution when input is passed to shell=True.	Safe command suffix proves command injection risk.
/hello?name=	Demonstrate unsafe dynamic template rendering risk.	Input is reflected through template rendering.
SQLite test data	Provide predictable rows for repeatable validation.	Multiple users returned during SQL injection demonstration.

Manual Validation Rules

- Use only local endpoints such as localhost and private Kind services.
- Redirect sensitive-file test output to /dev/null where possible; capture only proof of behavior, not secret content.
- Record the request, expected result, observed result, and screenshot reference for each test.
- Confirm scanner findings with manual evidence before documenting them as security issues.

Test Case	Procedure	Expected Result
Health Check	Request /health through curl.	Application returns a stable status response.
SQL Injection	Submit an OR 1=1 style payload through the id parameter.	Multiple rows are returned, proving unsafe query construction.
Command Injection	Append a safe command such as whoami to the host parameter.	Command output appears in the response, proving shell injection risk.
SAST Confirmation	Run Bandit against src/app.	Bandit flags subprocess/shell usage and related issues.
Dependency Confirmation	Run pip-audit against requirements.txt.	Vulnerable dependencies are reported and saved as evidence.



Evidence snapshot: SQL injection proof captured from the local vulnerable application.

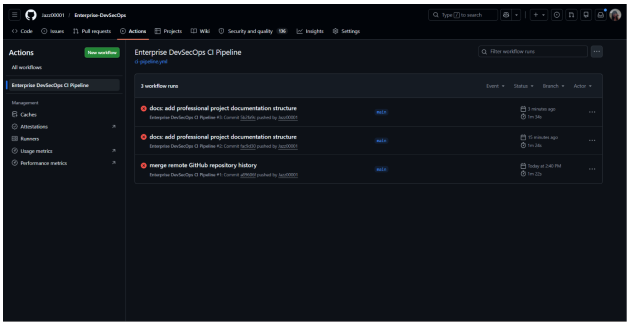
4. CI/CD Security and Container Hardening Methodology

The CI/CD methodology introduces automated security gates through GitHub Actions. The pipeline is designed to reflect a modern DevSecOps workflow: checkout, Python setup, SAST, dependency scanning, filesystem scanning, Docker build, image scanning, artifact upload, and registry push.

CI/CD Job	Tool or Action	Methodology Purpose
SAST	Bandit	Detect insecure Python coding patterns such as subprocess and shell injection risks.
SCA	pip-audit	Identify vulnerable Python dependencies from requirements.txt.
Filesystem Scan	Trivy fs	Scan repository contents for vulnerabilities, secrets, and misconfiguration indicators.
Docker Build	Docker	Build a reproducible application image from the hardened Dockerfile.
Image Scan	Trivy image	Scan the container image for operating system and dependency vulnerabilities.
SARIF Upload	GitHub Code Scanning	Publish machine-readable security results for review inside GitHub.
Registry Push	GHCR	Publish the image as a registry artifact for signing and SBOM attachment.

Container Hardening Controls

Control	Implementation	Security Value
Multi-stage build	Install dependencies in a builder stage and copy only required runtime packages.	Reduces final image complexity and build artifact exposure.
Non-root user	Run as fixed UID/GID 10001.	Limits impact if application code is compromised.
No login shell	Use non-login shell for app user.	Reduces interactive abuse potential.
Read-only app files	Set application ownership and restrictive permissions.	Supports Kubernetes readOnlyRootFilesystem.
Gunicorn runtime	Run Gunicorn instead of Flask debug server.	Better represents production-style serving.
Healthcheck	Call /health from inside the container.	Allows runtime status validation before Kubernetes deployment.



Evidence snapshot: GitHub Actions security pipeline running with automated security stages.

5. Kubernetes Hardening, RBAC, and Network Segmentation

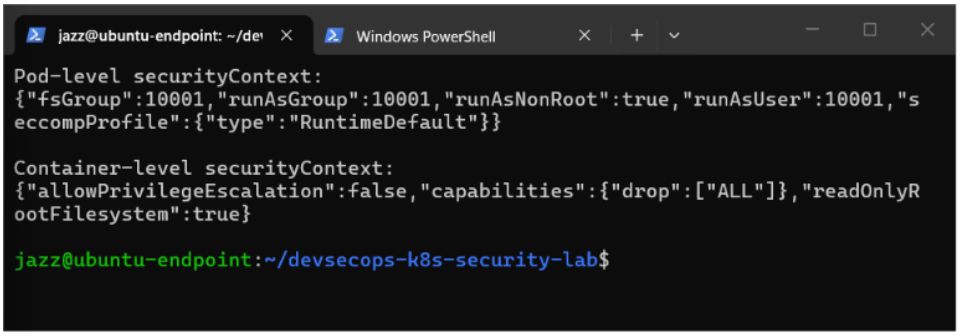
The Kubernetes deployment methodology converts the container into a hardened workload using explicit identity, runtime restrictions, resource controls, health probes, and network segmentation. The goal is to reduce blast radius if the application is compromised.

Kubernetes Control	Implementation	Reason
Restricted namespace	Pod Security labels enforce restricted standards.	Blocks risky pod settings at admission.
Dedicated ServiceAccount	Application uses demo-app-sa, not default identity.	Improves identity separation and auditability.
Token automount disabled	automountServiceAccountToken: false.	Reduces token theft risk inside the pod.
Non-root execution	runAsNonRoot, runAsUser 10001, runAsGroup 10001.	Prevents root execution inside the container.
Privilege controls	allowPrivilegeEscalation false, capabilities drop ALL.	Prevents escalation and removes unnecessary Linux capabilities.
Seccomp	RuntimeDefault profile.	Applies default syscall filtering.
Read-only root filesystem	readOnlyRootFilesystem true with /tmp emptyDir.	Blocks unauthorized writes to the app filesystem.
Resource controls	CPU and memory requests and limits.	Protects the small VM from resource abuse.
Health probes	Liveness and readiness probes use /health.	Improves runtime reliability and deployment status visibility.

Least-Privilege RBAC and NetworkPolicy

RBAC is validated using kubectl auth can-i. The methodology proves that the application identity can read only intended ConfigMaps and cannot read Secrets or list cluster Nodes. NetworkPolicy applies a default-deny model and then explicitly allows only trusted ingress and DNS egress.

Validation	Expected Result
can-i list configmaps as demo-app-sa	yes
can-i get secrets as demo-app-sa	no
can-i list nodes as demo-app-sa	no
blocked curl pod to demo-app service	timeout or denied
allowed curl pod to demo-app service	health response returned



Evidence snapshot: Kubernetes pod security context proof showing hardened runtime settings.

6. Admission Control, GitOps, and Drift Management

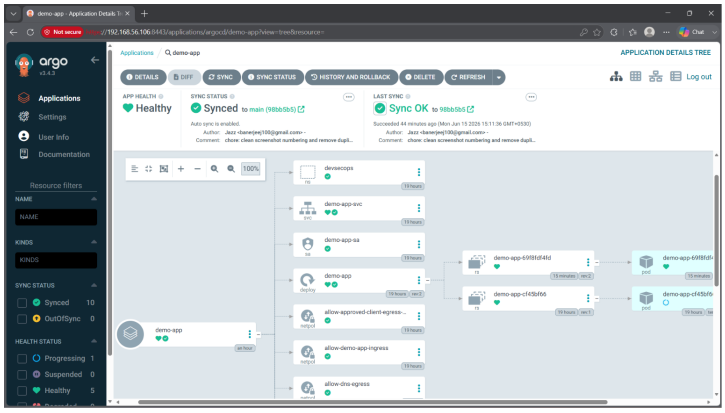
Admission control methodology adds policy enforcement before Kubernetes accepts resources. GitOps methodology then makes Git the desired state for cluster configuration, reducing manual drift and providing a clearer audit trail.

OPA Gatekeeper Methodology

Policy Area	Methodology
ConstraintTemplate	Define reusable Rego logic for required labels.
Constraint	Apply label requirements to Deployments in the devsecops namespace.
Required labels	security-owner, sbom, and signed.
Negative test	Attempt to create a Deployment without labels and verify admission denial.
Positive test	Apply a Deployment with required labels using server-side dry run and verify it is accepted.

ArgoCD GitOps Methodology

GitOps Area	Methodology
Repository source	Point ArgoCD to the project repository and k8s/base path.
Target cluster	Use the in-cluster Kubernetes API as the deployment target.
Automated sync	Allow ArgoCD to apply Git changes to the cluster.
Self-healing	Allow ArgoCD to revert live changes that do not match Git.
Pruning	Remove resources when they are removed from Git.
Drift demonstration	Manually scale the deployment and observe ArgoCD restore the Git-defined replica count.



Evidence snapshot: ArgoCD resource tree showing GitOps-managed Kubernetes resources.

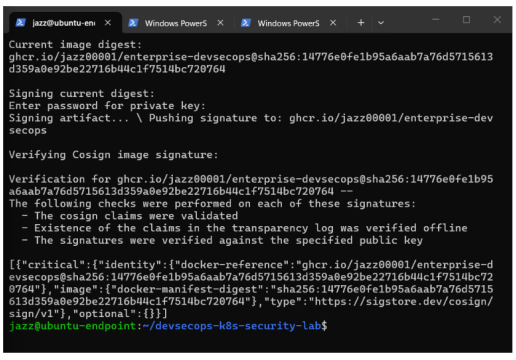
7. Supply Chain Security Methodology

Supply-chain security is implemented after the image is built and pushed to an OCI registry. The methodology demonstrates image signing, signature verification, SBOM generation, SBOM attachment, SBOM attestation, and SLSA-style provenance attestation.

Supply Chain Step	Tool	Methodology Outcome
Registry publish	GHCR	Image exists as a registry artifact that can be signed, scanned, and referenced.
Image signing	Cosign	Creates cryptographic proof that the image was signed with the project key.
Signature verification	Cosign	Confirms the public key can verify the image signature.
SBOM generation	Syft	Creates SPDX and CycloneDX software bill of materials files.
SBOM attachment	Cosign	Associates the SBOM with the image artifact in the registry.
SBOM attestation	Cosign	Signs the SBOM as a predicate tied to the image.
Provenance	Cosign + SLSA-style JSON	Documents source, builder context, commit, and image digest for learning purposes.
Admission metadata	Gatekeeper	Requires workloads to carry ownership and supply-chain labels.
Integrity Note	The SLSA-style provenance in this lab is generated locally for methodology demonstration. It should be described honestly as educational provenance, not full SLSA compliance from a trusted builder.	

Validation Commands Captured

```
cosign sign --key security/cosign/cosign.key IMAGE
cosign verify --key security/cosign/cosign.pub IMAGE
syft IMAGE -o spdx-json=security/sbom/sbom-spdx.json
cosign attest --key security/cosign/cosign.key --predicate security/sbom/sbom-spdx.json --type spdxjson
IMAGE
```



Evidence snapshot: Cosign image signature verification using the project public key.

8. Runtime Security and Red Team Simulation Methodology

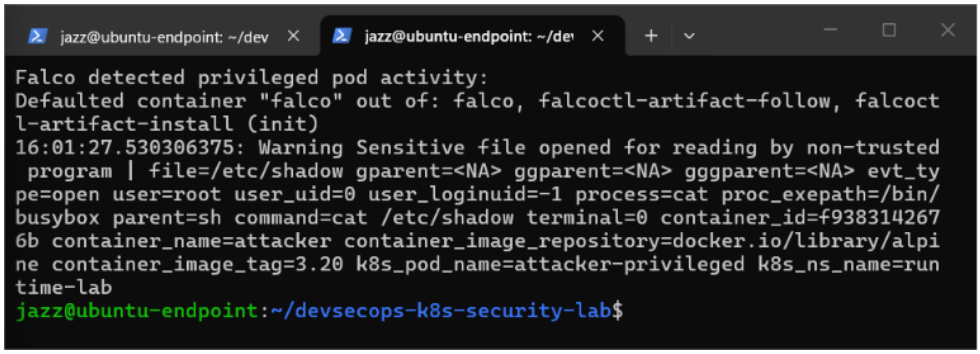
The runtime methodology demonstrates how dangerous Kubernetes configurations can be detected and then blocked. Falco is used to observe runtime behavior, while Pod Security and RBAC remediation reduce the ability to repeat the attack path.

Scenario	Attack Methodology	Detection or Remediation
Sensitive file access	Run a controlled pod that attempts to read /etc/shadow without printing secret content.	Falco logs and reports sensitive file access.

Scenario	Attack Methodology	Detection or Remediation
Privileged pod + hostPath	Deploy a lab-only privileged pod that mounts the node filesystem at /host.	Falco detects suspicious activity; Pod Security restricted later blocks the pod.
chroot demonstration	Use chroot /host to prove the node filesystem access pattern.	Evidence shows impact without modifying host files.
RBAC escalation	Bind a ServiceAccount to cluster-admin and test permissions with kubectl auth can-i.	Remove ClusterRoleBinding and replace with namespace Role/RoleBinding.
Retest	Repeat can-i checks and privileged pod apply after remediation.	Expected results change from yes/allowed to no/forbidden.

Runtime Safety Rules

- Do not print or store secret file contents; redirect sensitive output to /dev/null when triggering detection.
- Keep privileged pod demonstrations inside the Kind lab and delete attack pods immediately after evidence capture.
- Capture both detection evidence and prevention evidence so the report proves remediation, not only exploitation.
- Use kubectl auth can-i rather than stealing or printing service-account tokens.



Evidence snapshot: Falco detecting suspicious activity from the privileged pod simulation.

9. Security Observability with Prometheus and Grafana

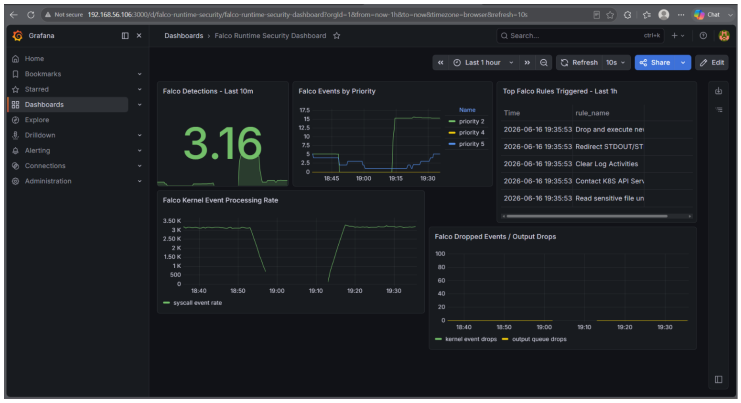
The observability methodology adds dashboards and metrics so runtime security events are not only logged but also measurable. Falco produces security events, Falcosidekick exposes event data, Prometheus scrapes metrics, and Grafana visualizes security posture.

Falco detects event	Falcosidekick exposes metrics	Prometheus scrapes metrics	Grafana shows dashboard	Rules alert on detections

Observability Component	Methodology Role
kube-prometheus-stack	Deploys Prometheus and Grafana in the monitoring namespace with lightweight settings for the VM.
Falco metrics	Enables rule counters and runtime security event metrics.
Falcosidekick	Forwards Falco events and exposes a Prometheus-readable metrics endpoint.
ServiceMonitor	Allows Prometheus Operator to discover and scrape Falco/Falcosidekick targets.
Custom dashboard	Shows Falco detections, priority distribution, triggered rules, event processing, and drop indicators.
PrometheusRule	Creates alerting logic for runtime detections and dropped events.

Key Metrics Reviewed

Metric / Query	Purpose
falcosecurity_falco_rules_matches_total	Counts triggered Falco rules.
sum by (priority) (increase(...[30m]))	Groups detections by priority.
topk(10, sum by (rule_name) (increase(...[1h])))	Shows most triggered Falco rules.
falcosecurity_scap_n_drops_total	Tracks dropped kernel events that may reduce detection quality.
falcosecurity_falco_outputs_queue_num_drops_total	Tracks dropped Falco output events.



Evidence snapshot: custom Falco Grafana dashboard for runtime security observability.

10. Finding Validation, Retesting, Reporting, and QA

The reporting methodology turns lab activity into professional security documentation. Each important issue is documented with description, severity, evidence, impact, remediation, and validation. The methodology separates scanner findings, manually confirmed findings, configuration risks, and remediated controls.

Report Element	Methodology Requirement
Finding title	Use clear titles that explain the weakness and affected area.
Severity	Assign severity using impact and exploitability in the lab context.
Affected component	Identify route, image, pod, namespace, RBAC object, policy, or pipeline stage.
Evidence	Reference screenshots, command output, YAML, scanner report, or dashboard metric.
Impact	Explain what could happen if the pattern existed in a real environment.
Remediation	Describe the exact control applied or recommended.
Retest result	Show that the risky behavior is blocked, reduced, or monitored after remediation.

Retesting Logic

Before Remediation	Control Applied	After Remediation
Privileged pod can mount host filesystem.	Pod Security restricted labels applied.	Privileged pod is rejected at admission.
ServiceAccount has cluster-admin.	ClusterRoleBinding removed; namespace RoleBinding created.	ServiceAccount cannot read secrets or create ClusterRoleBindings.
Manual scale creates GitOps drift.	ArgoCD self-heal enabled.	Replica count returns to Git-defined state.
Deployment can miss supply-chain labels.	Gatekeeper required-label policy applied.	Non-compliant Deployment is blocked.
Runtime events appear only in logs.	Prometheus and Grafana integration added.	Detections become queryable and dashboard-visible.

Quality Gate	A finding is considered report-ready only when the document includes technical evidence, business/security impact, remediation, and retest evidence. This avoids a tool-output-only portfolio and demonstrates real security engineering methodology.

PDF Quality Assurance

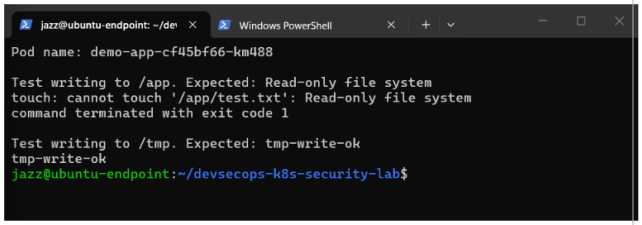
- Remove cover fields that are not requested and keep the title page clean.
- Use consistent margins, section spacing, table padding, and footer placement.
- Render pages to images and visually inspect every page for clipping, large unintended gaps, and overlap.
- Keep screenshots scaled and captioned so they support the methodology without crowding the text.

11. Evidence Snapshot Appendix

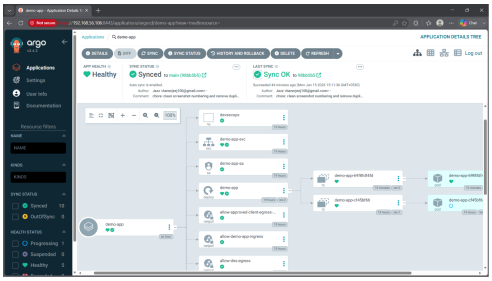
The selected screenshots below are included to improve readability and spacing while showing that the methodology is tied to real implementation evidence. The complete project archive contains the full screenshot set and generated security reports.

Bandit SAST findings from the vulnerable application.

Trivy image scan against the DevSecOps application image.



Read-only filesystem proof inside the hardened Kubernetes pod.



ArgoCD resource tree for GitOps-managed Kubernetes state.

SBOM attestation verification as supply-chain evidence.

RBAC remediation proof showing secrets access denied.

12. Reproducibility Checklist

This checklist summarizes the complete methodology in the order a reviewer or interviewer can discuss it. It also makes the project easier to reproduce in a private VM without needing to run every component at once.

Step	Reproducibility Check
1	Confirm VM resources, swap, Docker, kubectl, Helm, Kind, and basic tools.
2	Create repository structure and verify app code, Dockerfile, workflow, manifests, and docs.
3	Run the vulnerable Flask app locally and validate /health, SQL injection, and command injection safely.
4	Run Bandit, pip-audit, Trivy filesystem, Trivy config, and Trivy image scans.
5	Build the hardened Docker image, prove non-root UID, and load the image into Kind.
6	Apply hardened Kubernetes manifests and validate SecurityContext, probes, and service access.
7	Validate RBAC with kubectl auth can-i and NetworkPolicy with allowed and blocked test pods.
8	Install Gatekeeper and prove non-compliant Deployments are blocked.
9	Install ArgoCD and demonstrate sync, drift detection, self-healing, and pruning.
10	Push image to GHCR, sign with Cosign, generate Syft SBOM, and verify attestations.
11	Install Falco and simulate runtime activity; capture detection and remediation evidence.
12	Deploy Prometheus and Grafana; confirm Falco metrics, dashboard panels, and alert rules.
13	Write findings, map evidence, document remediation, and render final PDFs for visual QA.

Final Outcome	The completed methodology demonstrates end-to-end practical DevSecOps skills: application testing, secure CI/CD, Docker hardening, Kubernetes security, admission control, GitOps, supply-chain assurance, runtime detection, monitoring, and professional reporting.

Interview Talking Points

Topic	Strong Explanation
Why this lab matters	It shows the full path from vulnerable code to secure delivery, detection, remediation, and reporting.
What was hardened	The Docker image, Kubernetes workload, RBAC permissions, NetworkPolicy boundaries, admission rules, and deployment governance.
What was detected	Application flaws, dependency issues, container image vulnerabilities, Kubernetes misconfigurations, runtime events, and GitOps drift.
What was remediated	Privileged pod risk, overprivileged RBAC, missing supply-chain metadata, drift risk, and insufficient runtime visibility.
What makes it professional	Each phase includes evidence, validation, retesting, reporting, and honest lab-scope limitations.

End of methodology document.